

AegisDB: Software Architecture & Build Specification

An Embedded, Zero-Trust, Searchable Symmetric Encrypted (SSE) Storage Engine in Rust

Project Codename: AegisDB

Target Runtime: Rust 1.75+ / C-ABI

Doc Version: 1.0.0-SPEC

SYSTEM PROMPT / LLM AGENT INSTRUCTION

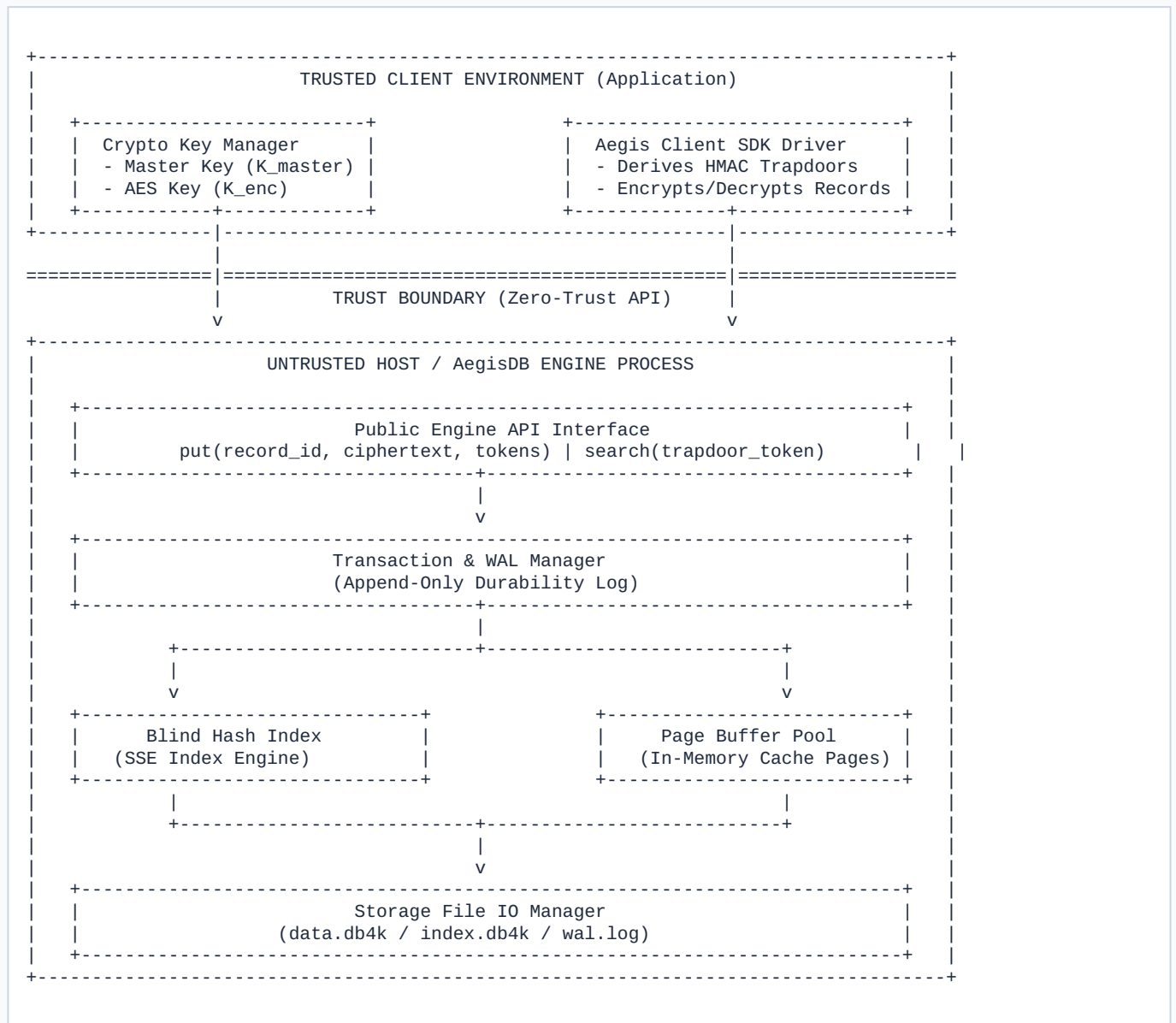
You are an expert systems software engineer and cryptographer. You are tasked with implementing **AegisDB**, an open-source, high-performance, embedded key-value storage engine written in Rust that supports native Searchable Symmetric Encryption (SSE). Follow this exact architecture, data layout, and trait contract specification.

1. Executive Summary & Vision

AegisDB addresses a critical security vulnerability in modern database engines: the cleartext exposure of memory and pages during query execution. Traditional databases decrypt pages in host RAM, leaving data vulnerable to hypervisor memory dumps, cold-boot attacks, or root process compromise. AegisDB provides an embedded storage engine—similar to RocksDB or SQLite—where data on disk and in host buffer memory remains strictly encrypted via AES-256-GCM.

Queries are executed using client-derived cryptographic search tokens (trapdoors) over structured blind hash indexes and order-revealing buckets without the host ever holding decryption keys or cleartext records.

2. Trust Model & System Boundaries



3. Cryptographic Primitives & Index Mathematics

The cryptography layer is split between client-side token derivation and server-side matching.

3.1 Client Key Derivation

From a master secret key K_{master} , the client uses HKDF (HMAC-based Extract-and-Expand Key Derivation Function) with SHA-256 to derive distinct sub-keys:

$$K_{enc} = \text{HKDF-Expand}(\text{HKDF-Extract}(\text{salt}, K_{master}), \text{"aegis-enc-v1"}, 32)$$
$$K_{search} = \text{HKDF-Expand}(\text{HKDF-Extract}(\text{salt}, K_{master}), \text{"aegis-search-v1"}, 32)$$

3.2 Payload Encryption

Data payloads are encrypted using **AES-256-GCM**. Each record payload generates a unique 96-bit (12-byte) initialization vector (nonce).

$$C = \text{AES-256-GCM-Encrypt}(K_{enc}, \text{Nonce}, \text{Plaintext}, \text{AssociatedData}=\text{RecordID})$$

3.3 Blind HMAC Trapdoor Token Generation

For a given query field value V (e.g., $V = \text{ext}\{ "123-45-6789" \}$), the client constructs an indexed search trapdoor token T using HMAC-SHA256:

$$T = \text{HMAC-SHA256}(K_{search}, \text{FieldID} \parallel "::" \parallel V)$$

4. Low-Level On-Disk Format Specifications

All page files are structured into fixed 4096-byte (4 KiB) pages to align with OS page cache and NVMe block sizes.

4.1 Binary Page Header Format (28 Bytes)

Offset (Bytes)	Field Name	Data Type	Description
0x00 - 0x03	Magic Bytes	u32 (4B)	Fixed Magic: 0x41454753 ("AEGS")
0x04 - 0x07	Page Type	u32 (4B)	0x01 = Data Page, 0x02 = Index Page, 0x03 = Free Page
0x08 - 0x0F	Page ID	u64 (8B)	Monotonically increasing page index number
0x10 - 0x17	LSN	u64 (8B)	Log Sequence Number for crash recovery validation
0x18 - 0x19	Slot Count	u16 (2B)	Number of tuple records stored in slot array
0x1A - 0x1B	Free Offset	u16 (2B)	Byte offset pointing to start of unallocated page space

4.2 Data Page Slotted Layout Diagram

```

+-----+
| PAGE HEADER (28 Bytes: Magic, Type, PageID, LSN, SlotCount, FreeOffset) |
+-----+
| Slot 0 Offset (u16) | Slot 1 Offset (u16) | Slot 2 Offset (u16) ... |
| ... (Slot offsets grow downwards ->) |
+-----+
|                                     |
|                               FREE PAGE SPACE                               |
|                                     |
+-----+
| (<-- Encrypted Record Data grows upwards) |
| Slot 1 Record: [RecordID (8B)][Nonce (12B)][Tag (16B)][Ciphertext...] |
| Slot 0 Record: [RecordID (8B)][Nonce (12B)][Tag (16B)][Ciphertext...] |
+-----+

```

5. Component Software Architecture & Trait Contracts

5.1 Core System Types (Rust)

```
// Types Definition Module
pub type RecordId = u64;
pub type PageId = u64;
pub type Lsn = u64;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
pub struct SearchToken(pub [u8; 32]);

#[derive(Debug, Clone)]
pub struct EncryptedRecordPayload {
    pub record_id: RecordId,
    pub nonce: [u8; 12],
    pub tag: [u8; 16],
    pub ciphertext: Vec<u8>,
}

#[derive(Debug, Clone)]
pub struct InsertTuple {
    pub record_id: RecordId,
    pub payload: EncryptedRecordPayload,
    pub search_tokens: Vec<SearchToken>,
}
```

5.2 Engine & Storage Interfaces

```
use std::path::Path;
use anyhow::Result;

pub trait StorageEngine: Send + Sync {
    /// Writes an encrypted record payload along with its blind trapdoor tokens
    fn insert(&mut self, tuple: InsertTuple) -> Result<()>;

    /// Searches the blind SSE index and returns all matching encrypted records
    fn search_by_token(&self, token: &SearchToken) -> Result<Vec<EncryptedRecordPayload>>;

    /// Retrieves a single record directly by RecordID
    fn get_by_id(&self, record_id: RecordId) -> Result<Option<EncryptedRecordPayload>>;

    /// Flushes dirty pages and WAL buffers to disk
    fn sync(&mut self) -> Result<()>;
}

pub trait ClientCryptoDriver {
    fn encrypt_record(&self, record_id: RecordId, plaintext: &[u8]) -> Result<EncryptedRecordPayload>;
    fn decrypt_record(&self, payload: &EncryptedRecordPayload) -> Result<Vec<u8>>;
    fn derive_trapdoor(&self, field_id: u16, value: &str) -> SearchToken;
}
```

5.3 Minimal Implementation Draft (Rust Engine Context)

```
use std::collections::HashMap;

pub struct MemoryIndexEngine {
    // Maps Blind Search Token -> List of Record IDs
    blind_index: HashMap<SearchToken, Vec<RecordId>>,
    // Maps Record ID -> Encrypted Payload
    data_store: HashMap<RecordId, EncryptedRecordPayload>,
    current_lsn: Lsn,
}

impl MemoryIndexEngine {
    pub fn new() -> Self {
        Self {
            blind_index: HashMap::new(),
            data_store: HashMap::new(),
            current_lsn: 0,
        }
    }
}

impl StorageEngine for MemoryIndexEngine {
    fn insert(&mut self, tuple: InsertTuple) -> Result<()> {
        self.current_lsn += 1;

        // Store payload
        self.data_store.insert(tuple.record_id, tuple.payload);

        // Update blind index maps without inspecting token contents
        for token in tuple.search_tokens {
            self.blind_index
                .entry(token)
                .or_insert_with(Vec::new)
                .push(tuple.record_id);
        }

        Ok(())
    }

    fn search_by_token(&self, token: &SearchToken) -> Result<Vec<EncryptedRecordPayload>> {
        let mut results = Vec::new();
        if let Some(ids) = self.blind_index.get(token) {
            for id in ids {
                if let Some(payload) = self.data_store.get(id) {
                    results.push(payload.clone());
                }
            }
        }
        Ok(results)
    }

    fn get_by_id(&self, record_id: RecordId) -> Result<Option<EncryptedRecordPayload>> {
        Ok(self.data_store.get(&record_id).cloned())
    }

    fn sync(&mut self) -> Result<()> {
        // In-memory reference sync stub
        Ok(())
    }
}
```

6. Implementation Roadmap & Phased Milestones

All agents or developers should implement this specification across four distinct phases:

Phase	Target Scope	Key Deliverables
Phase 1	Crypto Client Driver & Core Types	Implement HKDF key derivation, AES-256-GCM encryption wrappers, HMAC search token generators, and unit tests verifying zero plaintext leaks.
Phase 2	Slotted Data Page & File IO Engine	Build the 4 KiB binary page encoder/decoder, page buffer manager, and append-only write file interface with file locking.
Phase 3	SSE Index Manager & Buffer Pool	Implement the blind hash index, dynamic bucket splitting, and memory-mapped page cache LRU evictions.
Phase 4	WAL, Recovery & C-ABI Bindings	Write-Ahead Log recovery logic, crash safety checks, and expose FFI C-bindings (<code>`aegis_open`</code> , <code>`aegis_insert`</code> , <code>`aegis_search`</code>).